

Principles of I/O Software

1 Basic Idea

Definition

I/O software is the operating-system layer that hides device details and provides controlled, uniform, and reliable access to input/output devices.

- I/O hardware differs widely from device to device.
- I/O software gives programs a common way to use those devices.
- The OS handles device commands, errors, buffering, sharing, and blocking behavior.
- The goal is convenience for user programs without losing hardware control.

2 Main Goals

Goal	Meaning
Device independence	Programs should access I/O devices without knowing the exact device type.
Uniform naming	Files and devices should be named in a consistent way.
Error handling	Errors should be handled as close to the hardware as possible.
Synchronous interface	User programs should be able to use simple blocking I/O.
Asynchronous support	High-performance programs may need interrupt-driven or non-blocking I/O.
Buffering	The OS may need temporary storage between a device and the final destination.
Device sharing	The OS must handle both shared and dedicated devices correctly.

3 Device Independence

Core Principle

Device independence means a program should work with any suitable I/O device without being rewritten for that device.

A program that reads a file should not care whether the file is stored on a hard disk, DVD, USB stick, or network-backed storage. The program asks for file input; the operating system translates that request into the correct device-specific operation.

Example

The same file-reading program should work whether the input file is stored on a disk, removable drive, or other mounted storage device.

4 Uniform Naming

Definition

Uniform naming means files and devices are identified by simple names, such as strings or integers, instead of by physical device details.

In UNIX-like systems, devices and disks can be integrated into one file-system hierarchy. A USB stick can be mounted at `/usr/ast/backup`; copying to `/usr/ast/backup/monday` then copies data to the USB stick using an ordinary path name.

Command Example

`sort <input >output` should work no matter whether input comes from a disk file or keyboard, and output goes to a disk file or screen.

5 Error Handling

Principle

I/O errors should be handled at the lowest layer that can correctly fix them.

1. The device controller tries to correct hardware-level errors.
2. If the controller cannot fix the problem, the device driver retries or applies recovery.
3. Only if lower layers fail should upper layers be told about the error.

Layer	Error-handling responsibility
Controller	Detect and correct simple hardware errors when possible.
Device driver	Retry operations and apply device-specific recovery.
OS upper layers	Report failures only when low-level recovery fails.
User program	Usually sees only unrecoverable errors.

Transient Errors

Many I/O errors are temporary. A disk read error caused by dust may disappear when the block is read again.

6 Synchronous vs. Asynchronous I/O

Two Views

Physical I/O is often asynchronous, but user programs are easier to write when I/O appears synchronous.

Type	Meaning and use
Synchronous I/O	The calling program blocks until the operation finishes. This is common for ordinary file I/O.
Asynchronous I/O	The CPU starts the operation and continues. Completion is reported later by interrupt or event.

The OS often starts an interrupt-driven physical operation, blocks the user process, runs other work, and later wakes the process when the interrupt reports completion.

7 Buffering

Definition

Buffering means temporarily storing I/O data in memory before it reaches its final source or destination.

- A network packet may need to be stored before the OS knows its final destination.
- Audio output needs buffering so the device does not run out of data.
- Buffering decouples the rate at which data is produced from the rate at which it is consumed.
- Buffering can also reduce performance because it may require extra copying.

Performance Cost

Buffering improves correctness and timing, but extra memory copies can have a major impact on I/O performance.

8 Sharable and Dedicated Devices

Device Sharing

A **sharable device** can safely be used by multiple users or processes at the same time. A **dedicated device** must be assigned to one user or process until the operation is finished.

Device type	Example	OS concern
Sharable	Disk	Many users can have files open on the same disk at the same time.
Dedicated	Printer	Output from different users must not be mixed randomly on the same page.

Final Point

I/O software hides hardware differences while still handling errors, naming, buffering, blocking behavior, and device sharing correctly.